Gruber Login

Projektdokumentation · Login-Anwendung mit FastAPI, MySQL und Docker

1. Ziel des Projekts

Eine Webanwendung für Registrierung und Anmeldung, bei der Benutzer selbst wählen, wie stark ihr Login abgesichert ist. Das Projekt zeigt den Umgang mit Passwort-Hashing, Zwei-Faktor-Authentifizierung, Sessions und containerisiertem Betrieb.

2. Funktionsumfang

Modus	Ablauf
Passwort	E-Mail und Passwort, danach direkt angemeldet
E-Mail-Code	Passwort und sechsstelliger Code per E-Mail (10 Minuten gültig)
Authenticator (TOTP)	Passwort und Code aus einer Authenticator-App, Einrichtung per QR-Code (RFC 6238)

Weitere Funktionen: geschützte Willkommensseite (`/welcome`), Sessions mit sieben Tagen Laufzeit, Passwort-Reset per E-Mail-Code (15 Minuten gültig), optionale zeitliche Login-Beschränkung (konfigurierbar, z. B. 6–22 Uhr).

3. Technischer Aufbau

Komponente	Technik	Aufgabe
Backend	Python, FastAPI, SQLAlchemy (`app.py`)	Seiten ausliefern, API, Logik
Frontend	HTML, CSS, JavaScript (`templates/`, `static/`)	Oberfläche, SHA-256 im Browser
Datenbank	MySQL 8.4	Tabellen `users`, `sessions`, `email_otps`
Zugang	Cloudflare Tunnel (cloudflared)	HTTPS ohne Portfreigabe

Betrieb	Docker Compose	Start mit `docker compose up -d --build`

4. Ablauf der Anmeldung

1. Der Browser berechnet den SHA-256-Hash des Passworts, das Klartext-Passwort wird nicht übertragen.
2. Der Server prüft den Hash mit bcrypt gegen den gespeicherten Wert (`POST /api/login`).
3. Je nach Modus folgt direkt die Session oder die Abfrage des zweiten Faktors (`POST /api/verify-2fa`).
4. Der Server setzt einen zufälligen Session-Token als HttpOnly-Cookie. `/welcome` prüft ihn bei jedem Aufruf gegen die Datenbank.

5. Schnittstellen

Methode	Pfad	Zweck
POST	`/api/register`	Registrieren mit Modusauswahl
POST	`/api/login`	Login Schritt 1
POST	`/api/verify-2fa`	Login Schritt 2
POST	`/api/forgot-password`, `/api/reset-password`	Passwort zurücksetzen
GET	`/api/me`	Daten des angemeldeten Benutzers
POST	`/api/logout`	Abmelden

6. Inbetriebnahme

`cp .env.example .env`, dann `docker compose up -d --build db web`. Die Anwendung ist unter http://localhost:5000 erreichbar. Für den öffentlichen Zugang wird zusätzlich das Cloudflare-Tunnel-Token in der `.env` eingetragen. Ohne SMTP-Konfiguration werden E-Mail-Codes im Log ausgegeben (`docker compose logs web`). Die ausführliche Anleitung steht in der `README.md`.

7. Ressourcenbedarf (Schätzwerte, nicht gemessen)

Arbeitsspeicher insgesamt ca. 0,5–0,8 GB (MySQL ca. 400–600 MB, Web ca. 60–100 MB, cloudflared ca. 20–50 MB), CPU im Leerlauf praktisch ohne Last, Speicherplatz ca. 1 GB für die Images. Empfohlen: 1 vCPU, 2 GB RAM.

8. Sicherheit und bekannte Einschränkungen

Umgesetzt: bcrypt-Hashing auf dem Server, SHA-256 im Browser, zufällige Session-Tokens, HttpOnly-Cookie mit Secure-Flag (Standard, per `SESSION_COOKIE_SECURE` abschaltbar), Schutz vor Überschreiben fremder Konten bei erneuter Registrierung (Passwortprüfung), TOTP nach Standard, einheitliche Antwort beim Passwort-Reset, Routenschutz.

Bekannte Einschränkungen (Demo-Charakter des Projekts):

- Keine Begrenzung der Fehlversuche bei E-Mail- und Authenticator-Codes.
- Die Serverzeit des Containers (UTC) bestimmt die Login-Zeitsperre.
- Codes werden im Entwicklungsmodus im Klartext im Log ausgegeben.

Mögliche Erweiterungen: Rate-Limiting für Code-Eingaben, automatisierte Tests.

9. Aufgabenteilung

Person	Vortragsteil
Person A	Idee, technischer Aufbau, die drei Login-Modi
Person B	Ablauf, Sicherheit, Live-Demo, Fazit